Name: Sk Arshad Basha

Reg no: 192312336

1.Compare K-Nearest Neighbour and Case-Based Learning using a real-world example such as medical diagnosis. Analyze differences in memory usage, similarity measures, and prediction strategy.

Introduction

Both K-Nearest Neighbour (KNN) and Case-Based Learning (CBL) use previous examples to solve new problems. Consider a medical diagnosis system where historical patient records are stored with symptoms and diagnosed diseases.

For example, a new patient has:

- Fever
- Cough
- Sore throat
- High temperature

For example, fever may be given greater importance than a less significant symptom.

Prediction Strategy

KNN

If the three nearest patients have diagnoses:

Patient 1 → Flu

Patient 2 → Flu

Patient 3 → Cold

KNN predicts:

Flu

because Flu receives the majority vote.

Case-Based Learning

CBL retrieves the most similar historical case:

New Patient

↓

Find Similar Cases

↓

Retrieve Diagnosis

↓

Adapt Previous Diagnosis

↓

New Diagnosis

The previous case can be revised using additional information from the new patient.

Memory Usage

Both methods are memory-based, but their purpose differs.

KNN generally requires the feature vectors and labels of training examples to remain available for distance calculations.

CBL maintains a case base, which can contain detailed descriptions, previous solutions, outcomes, and contextual information.

Which is Better?

KNN is better when:

- The problem is straightforward classification.
- Data is well represented numerically.
- Fast implementation is required.

CBL is better when:

- Previous experience is important.
- Cases contain rich contextual information.
- Solutions need to be adapted rather than simply voted.

Conclusion

KNN classifies a new patient primarily through nearest-neighbour voting, while CBL retrieves and adapts previous medical experiences. CBL provides better explanation through previous cases, whereas KNN is simpler for direct classification.

2. Compare Locally Weighted Regression and Radial Basis Function networks for function approximation tasks. Use an example dataset to explain differences in locality, computational cost, and smoothness of predictions.

Introduction

Both Locally Weighted Regression (LWR) and Radial Basis Function (RBF) Networks can approximate nonlinear functions. Consider the dataset:
X=[0,1,2,3,4,5]
Y=[0,1,4,9,16,25]

This approximately represents:
y=x^2

If the query is:
xq=3

then points near 3 have greater influence than points near 0 or 5.

Thus, LWR creates a local model for every query.

RBF Working

An RBF network uses Gaussian functions:

where (c) is a centre and (\sigma) is the width.

For example:

Centres = [0, 2, 4]

Locality

LWR

Locality is explicitly determined at prediction time. Every new query calculates weights based on nearby training points.

RBF

Locality is represented by the Gaussian hidden units. The network learns a collection of local regions during training.

Computational Cost

LWR may become expensive because it repeatedly calculates distances and weighted regression for every query.

RBF networks have higher initial training cost, but prediction can be much faster once the network is trained.

Smoothness

LWR smoothness depends heavily on the bandwidth (\tau):

- Small (\tau) → very local, potentially irregular prediction.
- Large (\tau) → smoother but less local prediction.

For RBF:

- Small (\sigma) → narrow local responses.
- Large (\sigma) → smoother overlapping responses.

Conclusion

LWR is useful when the dataset is relatively small and local adaptation is important. RBF networks are preferable when repeated predictions are required because the learned network can provide fast nonlinear approximation.

3. Compare Naïve Bayes and Logistic Regression for a text classification task such as spam detection. Analyze assumptions about feature independence, probability estimation, computational efficiency, and performance on high-dimensional data. Discuss scenarios where one method outperforms the other.

Introduction

Consider a spam-detection system that classifies emails as:

- Spam
- Not Spam

Features can be extracted from email text using methods such as Bag-of-Words or TF-IDF.

Naïve Bayes

Naïve Bayes uses Bayes' theorem:

The weights determine how strongly individual words or features influence the spam probability.

Feature Independence

Naïve Bayes

Assumes that features are conditionally independent.

For example, it treats the presence of:

"free"

"prize"

as independent given the class, even though words may actually be related.

Logistic Regression

Does not require this independence assumption.

Therefore, Logistic Regression can model relationships between correlated features more effectively.

Probability Estimation

Naïve Bayes directly estimates class and feature probabilities from training data.

Logistic Regression estimates model parameters that produce class probabilities.

Computational Efficiency

Naïve Bayes is generally extremely fast to train and predict, making it suitable for large text datasets.

Logistic Regression requires iterative optimization, so training can be more computationally expensive.

High-Dimensional Data

Both methods can perform well with thousands or millions of text features.

Naïve Bayes is especially attractive when:

- The vocabulary is very large.
- Training data changes frequently.
- Fast training is required.

Logistic Regression can outperform it when:

- There is sufficient labelled data.
- Feature relationships matter.
- Good regularization is used.

Scenario Comparison

Naïve Bayes is preferable when:

- Training speed is important.
- Dataset is relatively small.
- Text contains many sparse features.
- A simple baseline is needed.

Logistic Regression is preferable when:

- More training data is available.
- Features are correlated.
- Stronger discriminative performance is required.
- Regularization can be applied.

Conclusion

Naïve Bayes is fast and effective for high-dimensional text classification, while Logistic Regression can achieve stronger performance when sufficient data and good feature engineering are available.

4. Analyze the differences between Support Vector Machines (SVM) and KNN in classification tasks. Use an example dataset to compare decision boundaries, scalability, sensitivity to noise, and computational complexity. Discuss how kernel functions in SVM improve performance for non-linear problems.

Introduction

Support Vector Machine (SVM) and K-Nearest Neighbour (KNN) are classification algorithms, but they use very different strategies.

Consider a dataset containing two classes:

Class A → SVM

Class B → KNN

SVM

SVM finds a hyperplane that separates classes while maximizing the margin.

The closest training points are called support vectors.

KNN

KNN does not explicitly learn a decision boundary.

For a new point:

1. Calculate distance to training points.
2. Find K nearest points.
3. Use majority voting.

Example

Suppose:

K = 3

Nearest neighbours:

Class A

Class A

Class B

KNN predicts:

Class A

SVM instead uses its learned decision boundary to determine which side contains the new point.

Scalability

KNN can become slow during prediction because it may need to calculate distances to many training points.

SVM has a more expensive training phase, but prediction is generally much faster once the model is trained.

For very large datasets, linear SVM can be more scalable than basic KNN.

Sensitivity to Noise

KNN can be sensitive to noisy or incorrectly labelled neighbours.

For example:

A A B A

If the noisy point B is close to a new point, it can influence the prediction.

SVM uses the margin and regularization to reduce the impact of individual observations, although it can still be affected by significant outliers.

Kernel Functions

A linear SVM can only create a linear boundary.

For nonlinear data, kernel functions can map the data into a higher-dimensional feature space.

Common kernels include:

- Linear
- Polynomial
- Radial Basis Function (RBF)

Which is Better?

SVM:

- Better for high-dimensional datasets.
- Effective with clear margins.
- Good for nonlinear problems using kernels.
- Suitable when fast prediction is important.

KNN:

- Very simple.
- No complicated training.
- Good for small datasets.
- Useful when local similarity is meaningful.

Conclusion

SVM learns an explicit decision boundary, whereas KNN classifies using nearby examples. SVM generally scales better for prediction and high-dimensional data, while KNN is simpler and useful for smaller datasets.

5. Compare K-Means Clustering and Hierarchical Clustering using a dataset with unknown structure. Analyze differences in cluster formation, computational cost, scalability, and interpretability. Discuss how each method handles varying cluster shapes and the impact of choosing the number of clusters.

Introduction

Both K-Means Clustering and Hierarchical Clustering are unsupervised learning algorithms. They group unlabeled data into clusters.

Consider a dataset containing customer information such as:

- Age
- Annual income
- Spending score

The objective is to identify groups of similar customers.

K-Means Working

K-Means follows these steps:

Choose K

↓

Initialize Centroids

↓

Assign Points to Nearest Centroid

↓

Calculate New Centroids

↓

Repeat

↓

Final Clusters

The most common approach is agglomerative clustering:

Each point starts as a cluster

↓

Find closest clusters

↓

Merge them

↓

Repeat

↓

Dendrogram

The dendrogram can then be cut at a selected level to obtain the desired number of clusters.

Cluster Formation

K-Means

K-Means directly divides the dataset into K groups.

It attempts to minimize within-cluster variation.

Hierarchical

Hierarchical clustering creates a tree showing relationships between observations.

This provides more information about how clusters are related.

Computational Cost

K-Means is generally computationally efficient and suitable for large datasets.

Hierarchical clustering typically requires considerably more computation and memory, especially as the number of observations grows.

Therefore:

Large dataset → K-Means is usually preferred.

Small/medium dataset requiring hierarchical interpretation → Hierarchical clustering can be preferred.

Scalability

K-Means scales relatively well because it repeatedly calculates distances to a limited number of centroids.

Hierarchical clustering can become expensive because many pairwise distances may need to be considered.

Interpretability

K-Means provides final cluster assignments and centroids.

Hierarchical clustering provides a dendrogram, which makes it easier to understand relationships between groups.

Varying Cluster Shapes

K-Means generally works best when clusters are:

- Compact
- Well separated
- Approximately spherical

It may perform poorly when clusters are elongated or have irregular shapes.

Hierarchical clustering can handle some more complex structures depending on the linkage method.

For example:

- Single linkage can connect elongated structures.
- Complete linkage tends to produce compact clusters.
- Average linkage provides a compromise.

Methods include:

- Elbow method
- Silhouette score
- Domain knowledge

Hierarchical

The number of clusters does not need to be fixed at the beginning. The dendrogram can be cut at different levels to obtain different numbers of clusters.

Advantages

K-Means

- Simple.
- Fast.
- Scalable.
- Easy to implement.
- Effective for compact clusters.

Hierarchical

- Dendrogram provides useful interpretation.
- Does not require initial K.
- Shows relationships between clusters.
- Useful for exploratory analysis.

Limitations

K-Means

- Requires K.
- Sensitive to initialization.
- Sensitive to outliers.
- Not suitable for all cluster shapes.

Hierarchical

- Computationally expensive.
- Less suitable for very large datasets.
- Once a merge is made, basic hierarchical methods generally cannot undo it.

Conclusion

K-Means is generally better for large datasets and compact clusters, while Hierarchical Clustering is useful for exploring unknown structure and understanding relationships between groups. K-Means requires the number of clusters to be chosen, whereas hierarchical clustering provides a dendrogram that allows the cluster count to be selected afterward.